

```

    if (n % 2 === 0) {
        // n is even.
        opStack.push("square");
        n = Math.floor(n / 2);
    } else if (n % 2 === 1) {
        // n is odd.
        n -= 1;
        opStack.push("multiply");
    }
}

// Step 2: Perform the operations in reverse order.
let result = a; // Start result at `a`.
while (opStack.length > 0) {
    let op = opStack.pop();

    if (op === "multiply") {
        result = result * a;
    } else if (op === "square") {
        result = result * result;
    }
}

return result;
}

document.write(exponentWithPowerRule(3, 6) + "<br />");
document.write(exponentWithPowerRule(10, 3) + "<br />");
document.write(exponentWithPowerRule(17, 10) + "<br />");
</script>

```

Our algorithm keeps reducing n by dividing it in half (if it's even) or subtracting 1 (if it's odd) until it is 1. This gives us the squaring or multiply-by- a operations we have to perform. After finishing this step, we perform these operations in reverse order. A generic stack data structure (separate from the call stack) is useful for reversing the order of these operations since it's a first-in, last-out data structure. The first step pushes squaring or multiply-by- a operations to a stack in the `opStack` variable. In the second step, it performs these operations as it pops them off the stack.

For example, calling `exponentWithPowerRule(6, 5)` to calculate 6^5 sets a as 6 and n as 5. The function notes that n is odd. This means we should subtract 1 from n to get 4 and push a multiply-by- a operation to `opStack`. Now that n is 4